

Can the SPO tree find a planted threshold?

A controlled test on daily data with market-sized parameters

22 September 2026

In short. We build an artificial market in which one daily feature x decides the regime of an asset: the price drifts up while x is above a threshold d and down while it is below. The tree trades the asset against cash, sees x alone, and decides every h days. Because we planted the threshold, we know the answer and can measure how close the tree comes. Everything is sized on daily markets: 1% volatility a day, a regime drift of 5 to 25% a year, a feature that mean-reverts over weeks, 15 years of training data, 3 basis points of fees.

Results (Section 4, 2,000 fitted trees).

- With a regime drift of $\pm 25\%$ a year the tree finds the threshold at every trading rate, from daily to monthly: its split is on the planted value to within ± 0.25 standard deviations of x in 60 to 95% of the datasets, and it earns 75 to 85% of what the ideal rule earns out of sample.
- With $\pm 13\%$ a year it still finds it in 62 to 82% of the datasets, less precisely, and captures 25 to 50% of the gain. With $\pm 5\%$ a year nothing can be found in 15 years of data, and the tree mostly keeps no split, which is the right answer: the ideal rule itself earns almost nothing there.
- Daily to weekly decisions are equivalent. Monthly decisions lose precision and profit, all the more when the feature is fast (half-life of a week): the regime then changes inside the holding period.
- A defect was found and fixed on the way: a tree that stayed in cash had an undefined Sharpe ratio, which made pruning reject every split whose alternative was cash (Section 6).
- *Limit*: one clean feature. The earlier experiment (Section 5) showed decoys are ignored at a huge drift; whether they stay ignored at these drifts is the next test.

1 The model under test

Decision. Two assets, the risky asset and cash. At a decision the portfolio $w = (w_1, w_2)$, $w \geq 0$, $w_1 + w_2 = 1$, is chosen from the current holdings u by the mean-variance problem with proportional costs (Wang & Hasuiké, “DIR”, eq. 8):

$$w^*(\hat{c}; u, \Sigma) = \arg \max_{w \in \Delta} \hat{c}^\top w - f^\top |w - u| - \lambda w^\top \Sigma w, \quad f = (3 \text{ bp}, 0), \quad \lambda = 1, \quad (1)$$

where $\hat{c}$ is a vector of scores (one per asset) supplied by the predictor and Σ the covariance of the returns until the next decision. The problem is solved exactly (enumeration of the KKT cases).

SPO loss. With the realised returns r , the utility of a portfolio is $U(w; r, u, \Sigma) = r^\top w - f^\top |w - u| - \lambda w^\top \Sigma w$ and the *decision regret* of the scores $\hat{c}$ is

$$\ell(\hat{c}) = U(w^*(r; u, \Sigma)) - U(w^*(\hat{c}; u, \Sigma)) \geq 0, \quad (2)$$

the utility lost by deciding on $\hat{c}$ instead of on the returns that happened (the SPO loss of Elmachetoub & Grigas with costs and risk inside). The predictor is trained on this regret, not on a forecast error.

SPO tree. The predictor is a decision tree on x of depth at most 2, each leaf holding one score vector $\hat{c}_\ell = (\hat{c}_\ell, 0)$ (cash has score 0). It is fitted as in SPOT (Elmachtoub, Liang & McNellis), with two additions of this implementation.

- *Growth*, on the first 75% of the training decisions. At each round and for every leaf, every threshold on x is screened, the three best are refitted (leaf scores by a bounded coordinate search on the empirical regret, $|\hat{c}_\ell| \leq 10\%$, at least 20 decisions per leaf), and the split with the largest total regret reduction is accepted if the replayed training Sharpe ratio rises. The holdings u_t in (2) are those of the tree’s own replayed path: fees depend on what was held before.
- *Pruning*, on the last 25% (the “checking” decisions, unseen during growth): a split stays only if, there, it lowers the mean regret and raises the Sharpe ratio. Leaf scores are then refitted on all training decisions.

The optimizer, the tree and its settings are those of `run.py` (the real-data pipeline).

Two clocks. Data are daily; decisions are taken every h days and held in between. A training row is one decision: x_t on the decision day, the label $r_t = S_{t+h}/S_t - 1$ for the asset (0 for cash) and $\Sigma_t = \text{diag}(h \hat{\sigma}_t^2, 10^{-6})$ with $\hat{\sigma}_t^2$ the variance of the last 180 daily returns. h is the trading rate.

2 The planted world

The feature is a daily Ornstein–Uhlenbeck process and the asset a geometric random walk whose drift follows the feature (`synthetic_data.py`):

$$\begin{aligned} x_{t+1} &= m + e^{-k}(x_t - m) + \eta_t, & \eta_t &\sim \mathcal{N}(0, s^2(1 - e^{-2k})), \\ \log \frac{S_{t+1}}{S_t} &= \mu_t - \frac{1}{2}\sigma^2 + \sigma\varepsilon_t, & \mu_t &= \begin{cases} +\mu & x_t \geq d \\ -\mu & x_t < d \end{cases}, \quad \mu = e\sigma. \end{aligned} \quad (3)$$

x has long-run mean m , stationary variance s^2 and half-life $\ln 2/k$ days. The *edge* $e = \mu/\sigma$ is the daily drift-to-volatility ratio, so $e\sqrt{252}$ is the annual Sharpe ratio of a trader who knows the regime every day. The threshold is planted **at the mean of the feature**, $d = m$.

What a correct tree should find. A decision is held for h days while x keeps moving, so what matters at a decision is the expected return over the holding period given x now. With $B_h = \sum_{j=0}^{h-1} \text{sign}(x_j - d)$ the number of days spent above d minus the number below,

$$g_h(x) = \mathbb{E}\left[e^{\mu B_h} \mid x_0 = x\right] - 1 \quad (5)$$

(the price noise averages out), computed by Monte Carlo on (3). Because $d = m$, the law of $x - m$ is symmetric and $g_h(m + z) = -g_h(m - z)$ up to the convexity of the exponential (negligible here): **g_h changes sign at d for every holding period.** By (1) the ideal position is the full asset where $g_h(x) > 2\lambda h\sigma^2$ (0.02% for $h = 1$, 0.42% for $h = 21$) and cash below, so the decision changes at d up to a few hundredths. The right threshold is therefore d at every trading rate. (When $d \neq m$ this is no longer true: see Section 5.)

Parameter	Base	Also tested	Meaning and realism
σ	1% a day	–	volatility of the asset, 16% a year (an equity index)
$e = \mu/\sigma$	0.10	0.02, 0.05	drift $\pm 25\%$ a year ($\pm 5\%$, $\pm 13\%$); Sharpe of a perfect daily timer $e\sqrt{252} = 1.6$ (0.3, 0.8)
k	0.02 a day	0.01, 0.05, 0.10	reversion speed: half-life 35 days (69, 14, 7 days)
m, s^2, d	0, 1, 0	–	x is a standardized feature; the threshold sits at its mean
h	5 days	1, 2, 10, 21	trading rate: weekly (daily, twice a week, every two weeks, monthly)
training, test	15 years, 5 years	–	3,780 and 1,260 daily observations; 180 warm-up days for Σ
fee, λ	3 bp, 1	–	per traded notional on the asset, 0 on cash
tree	depth ≤ 2 , ≥ 20 per leaf	–	all thresholds screened, shortlist 3, last 25% of the training decisions prune
datasets per cell	100	50	100 at $e \in \{0.05, 0.10\}$, 50 at $e = 0.02$ and in the k sweep

Table 1: Parameters (the defaults of `Case` in `recovery_test.py` and the sweeps of `recovery_figures.py`). One decision every h days over 15 years gives 3,780, 1,890, 756, 378 and 180 training decisions for $h = 1, 2, 5, 10, 21$ (the last quarter for pruning), and 1,260, 630, 252, 126, 60 test decisions.

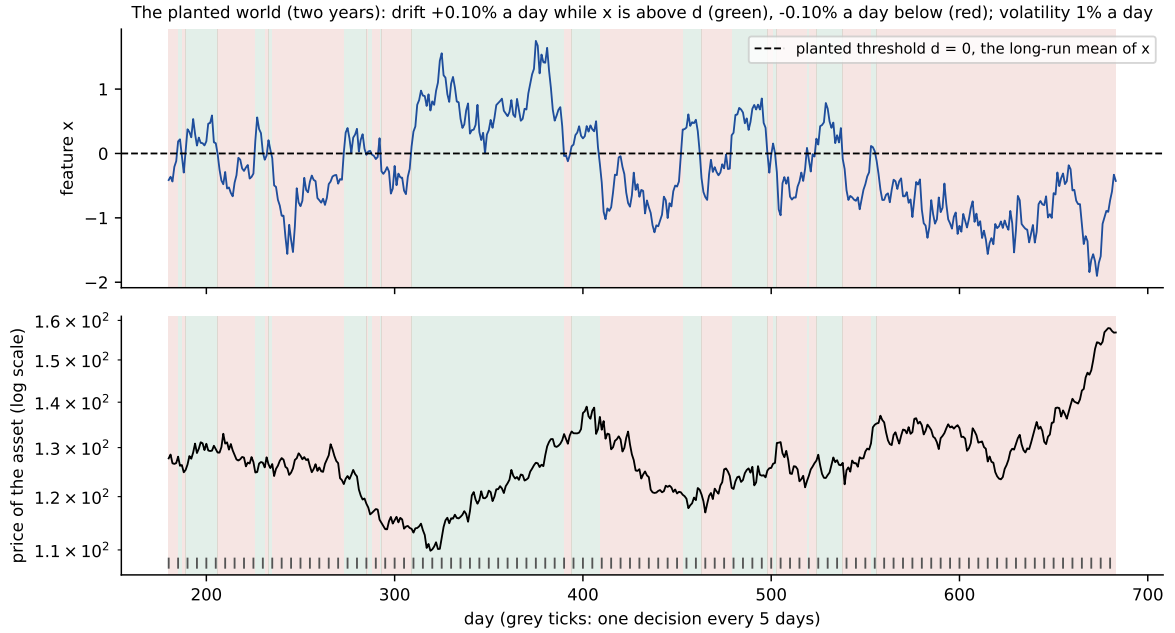


Figure 1: Two years of one dataset (base parameters, weekly decisions). The background is the true regime, which the tree never sees; a decision is taken at each grey tick and held until the next one.

3 Protocol

For each cell (a trading rate h with an edge e , or with a speed k at $e = 0.10$) and each of its N independent datasets: simulate $180 + 3,780 + 1,260$ days; fit the tree on the 15 training years; replay it on the 5 test years, starting from its holdings at the end of training. Four benchmarks are replayed on the same rows with the same optimizer: the *ideal rule* (optimizer fed with $\hat{c} = (g_h(x), 0)$, the best possible use of x); the *rule at d* (scores $\pm 10\%$ according to $x \geq d$); *always the asset*; the *tree without splits* (one leaf, i.e. no information). Four measurements:

1. **Does it split, and where?** Share of datasets whose first split is on x (with one feature, the alternative is no split); the distribution of the first threshold, whose target is $d = 0$ in units of the standard deviation of x .
2. **Test Sharpe ratio**, annualized as $\sqrt{252/h} \times$ the per-decision Sharpe of net returns, averaged over the datasets.
3. **Share of the possible gain captured:** $(R_{\text{tree}} - R_{\text{flat}})/(R_{\text{ideal}} - R_{\text{flat}})$ with R the mean test return, pooled over the datasets of a cell (blank when the ideal rule gains under 0.5% a year: nothing to capture).
4. **No split kept:** share of datasets where pruning left a single leaf.

4 Results

4.1 One dataset

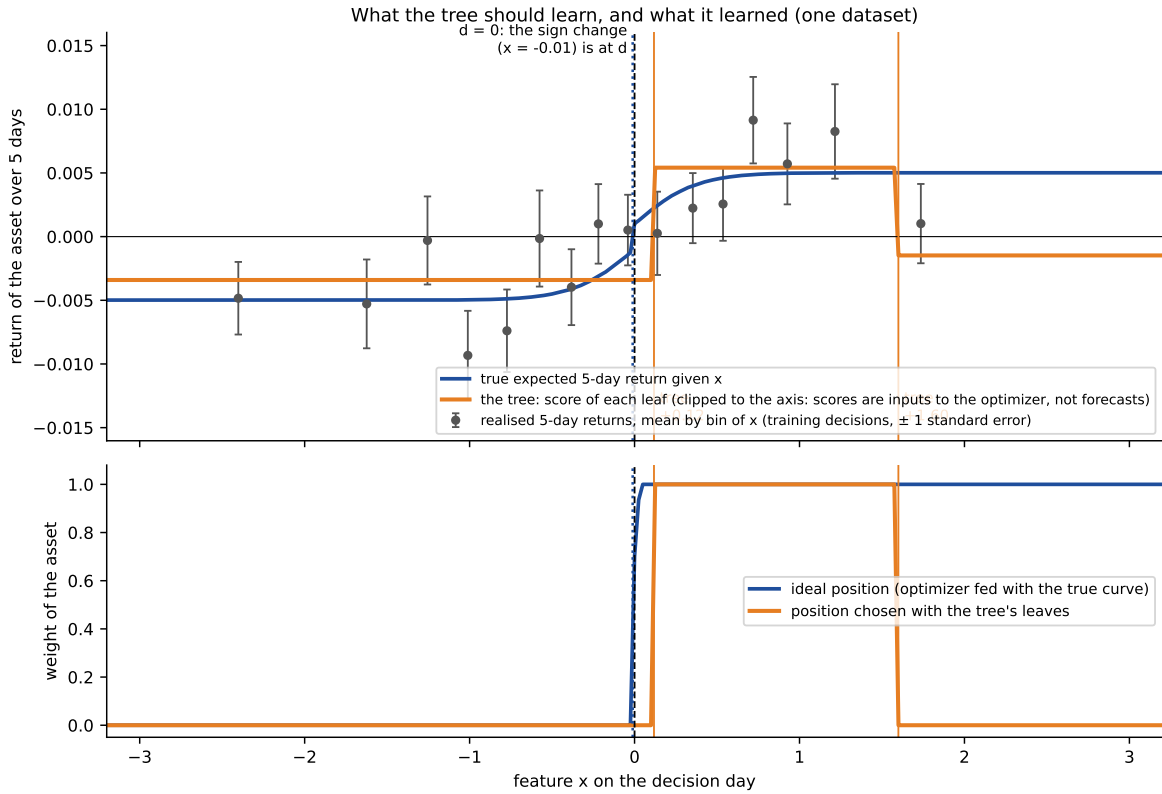


Figure 2: Base parameters, weekly decisions, dataset 42. *Top:* realised 5-day returns by bin of x (training decisions), the true $g_5(x)$ of (5), and the tree’s leaf scores. *Bottom:* the position the optimizer takes with the true curve and with the tree. Growth made three splits. Pruning removed the one at $x \leq -2.01$ and kept the root split at $x \leq 0.12$ (on the checking decisions: regret $0.0097 \rightarrow 0.0066$ per decision, Sharpe $0.06 \rightarrow 0.20$) together with a second-level one at $x \leq 1.60$ (31 decisions, regret $0.0067 \rightarrow 0.0066$), an example of the tail leaves discussed in Section 7.

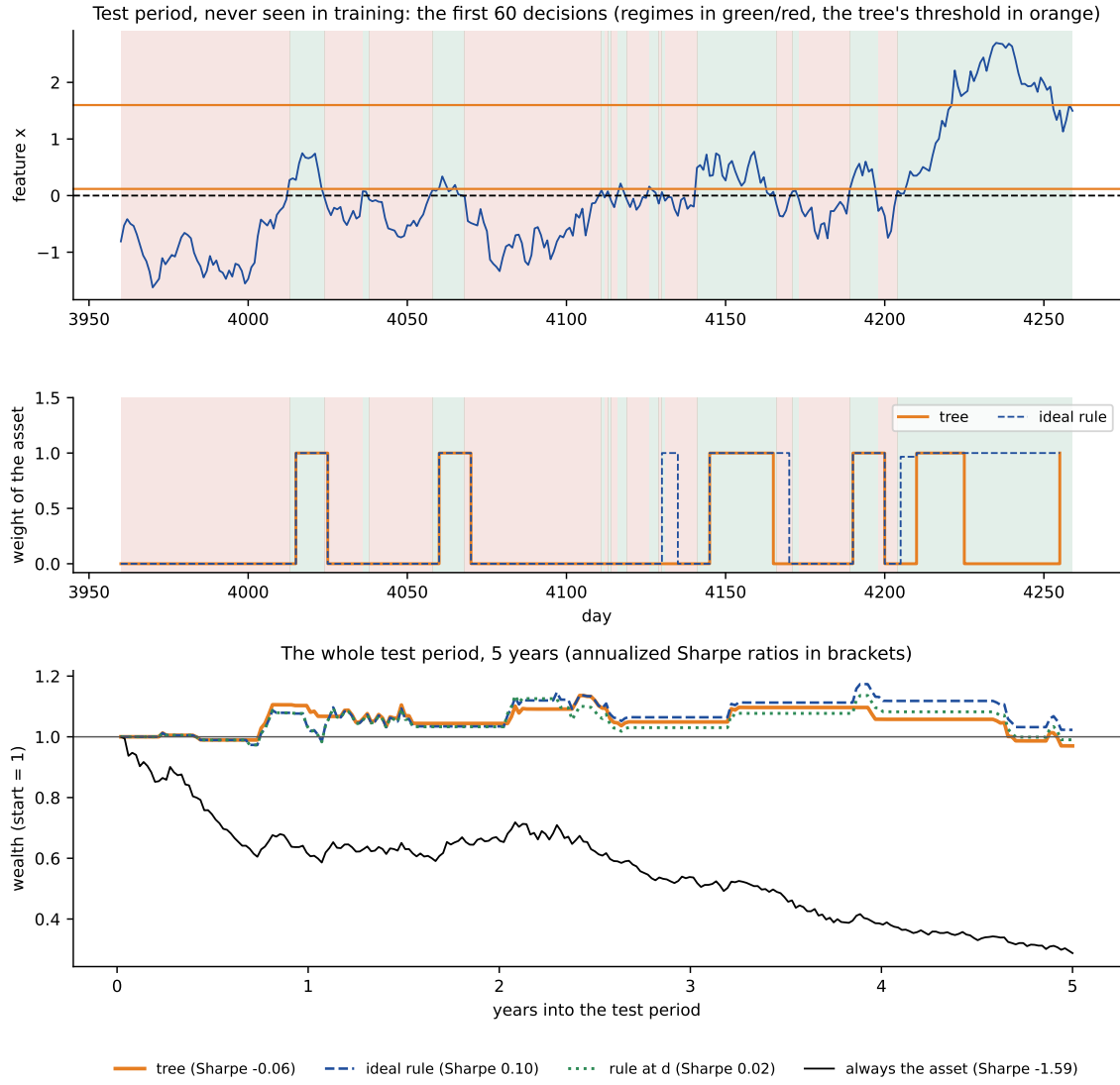


Figure 3: The same dataset out of sample. *Top, middle:* the first 60 test decisions; the tree is in cash below its threshold and fully invested above, like the ideal rule. *Bottom:* wealth over the 5 test years. This test window is dominated by the down regime (the asset loses 22% a year), so every rule earns little; what the figure shows is the decisions, not the level of the returns.

4.2 Where the threshold lands, over 100 datasets per cell

Where the tree puts its thresholds on x ($k = 0.02$, 100 datasets per panel; x has standard deviation 1)

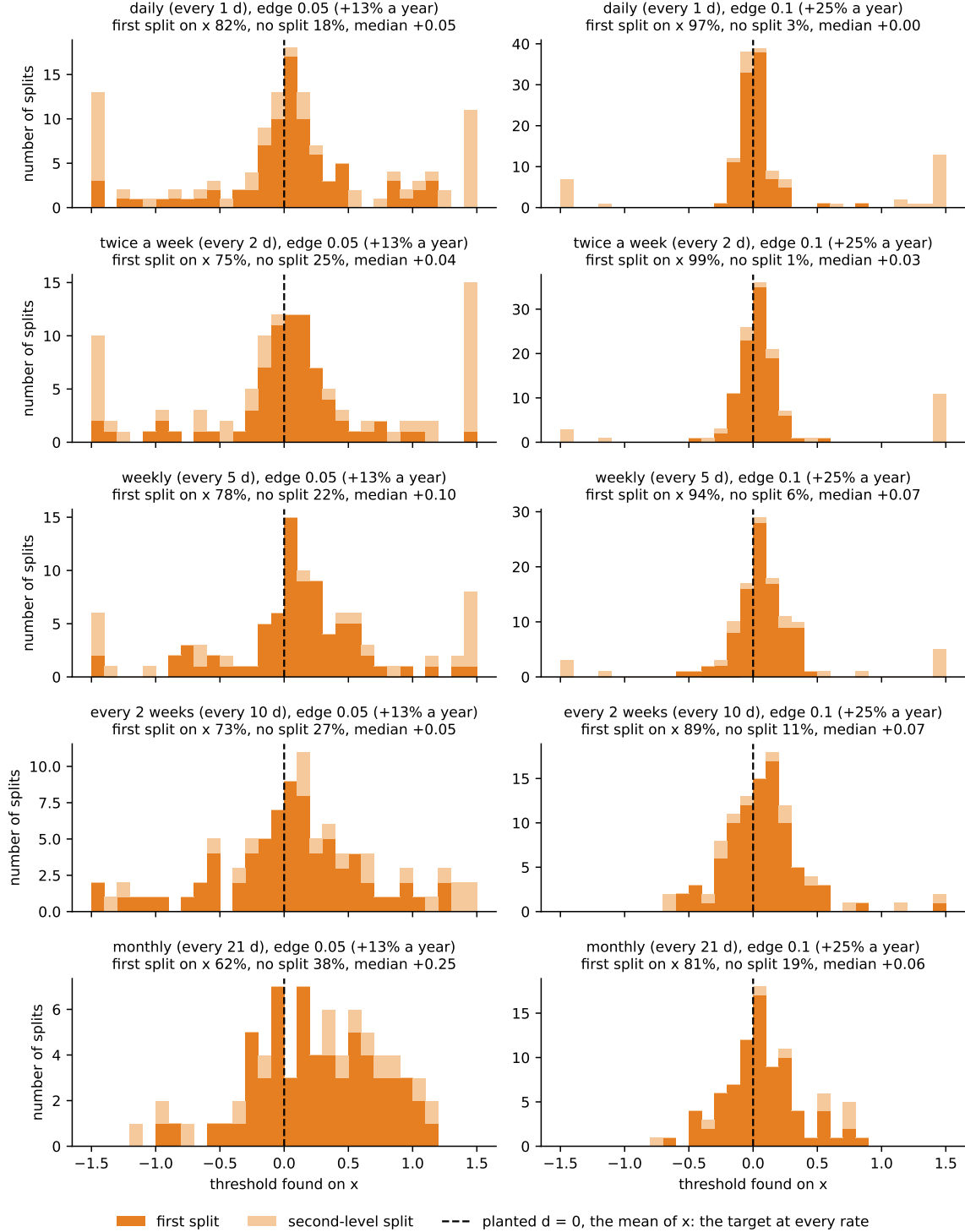


Figure 4: All thresholds on x kept in the final trees, one panel per trading rate, $k = 0.02$; *top*: edge 0.05, *bottom*: edge 0.10. The target is $d = 0$ at every rate; x has standard deviation 1. Bars at ± 1.5 collect thresholds beyond the axis (tail leaves).

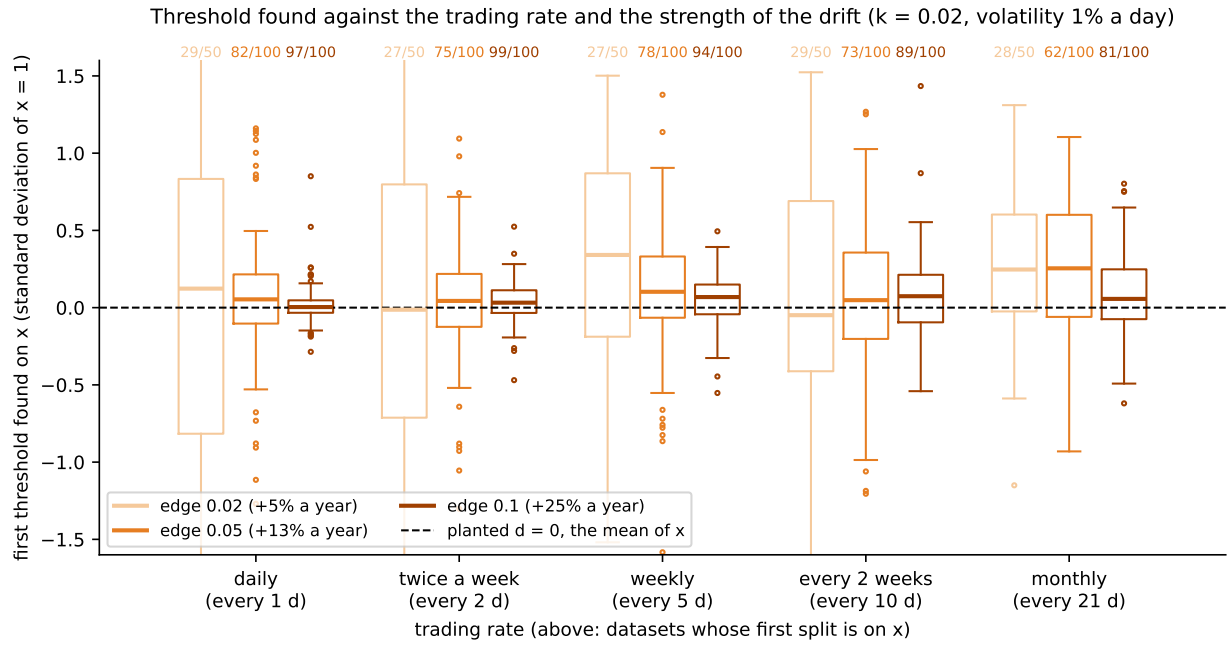


Figure 5: First threshold found, by trading rate and edge (boxes: quartiles; the count above each box is the number of datasets that split).

4.3 Trading rate against edge, and against the speed of the feature

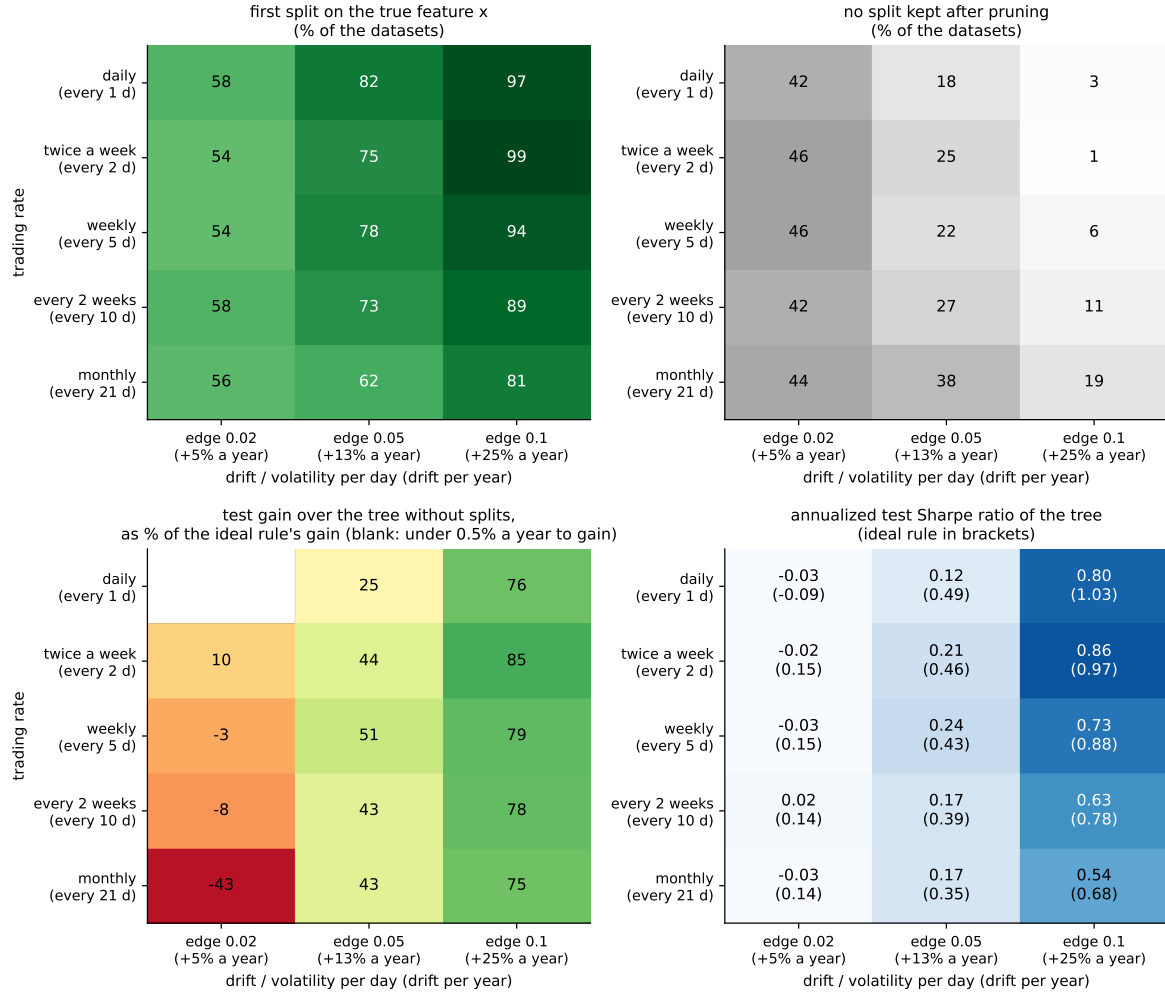


Figure 6: Rows: trading rate; columns: edge ($k = 0.02$). From left to right: datasets whose first split is on x ; datasets that keep no split; share of the ideal rule's test gain captured; annualized test Sharpe ratio of the tree (ideal rule in brackets).

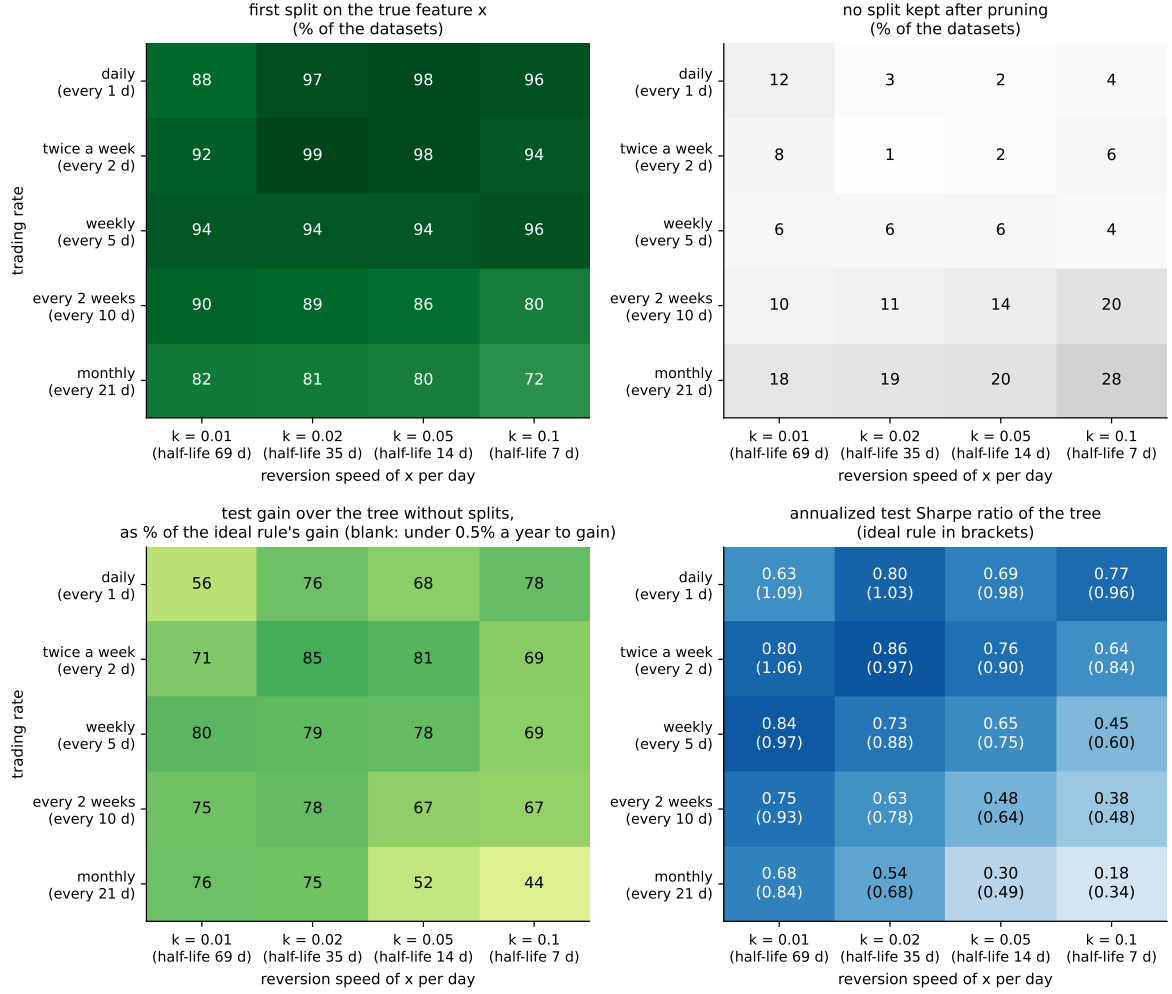


Figure 7: Same maps with the reversion speed of x in columns, at edge 0.10.

edge	h	splits (%)		first threshold on x			annualized test Sharpe			gain
		on x	none	median [quartiles]	$ \cdot \leq 0.25$		tree	ideal	at d	captured
0.10	1	97	3	+0.00 [−0.03, +0.05]	95%		0.80	1.03	1.04	76%
	2	99	1	+0.03 [−0.03, +0.11]	92%		0.86	0.97	0.98	85%
	5	94	6	+0.07 [−0.04, +0.15]	81%		0.73	0.88	0.88	79%
	10	89	11	+0.07 [−0.10, +0.21]	70%		0.63	0.78	0.78	78%
	21	81	19	+0.06 [−0.07, +0.25]	60%		0.54	0.68	0.66	75%
0.05	1	82	18	+0.05 [−0.10, +0.22]	60%		0.12	0.49	0.49	25%
	2	75	25	+0.04 [−0.12, +0.22]	64%		0.21	0.46	0.46	44%
	5	78	22	+0.10 [−0.07, +0.33]	53%		0.24	0.43	0.42	51%
	10	73	27	+0.05 [−0.20, +0.36]	47%		0.17	0.39	0.38	43%
	21	62	38	+0.25 [−0.06, +0.60]	34%		0.17	0.35	0.34	43%
0.02	1	58	42	+0.12 [−0.82, +0.83]	24%	−0.03	−0.09	0.14		—
	2	54	46	−0.01 [−0.71, +0.80]	15%	−0.02	0.15	0.13		10%
	5	54	46	+0.34 [−0.19, +0.87]	26%	−0.03	0.15	0.13		−3%
	10	58	42	−0.05 [−0.41, +0.69]	24%	0.02	0.14	0.13		−8%
	21	56	44	+0.25 [−0.02, +0.60]	32%	−0.03	0.14	0.14		−43%

Table 2: Trading rate h (days between decisions) against edge, $k = 0.02$; 100 datasets per row at edges 0.10 and 0.05, 50 at 0.02. “Always the asset” has an annualized Sharpe between -0.04 and 0.00 everywhere (no unconditional drift). Standard error of the tree–ideal Sharpe difference: 0.03 to 0.05.

half-life of x	h	splits (%)		first threshold on x		annualized Sharpe		gain
		on x	none	median [quartiles]	$ \cdot \leq 0.25$	tree	ideal	captured
69 days ($k = 0.01$)	1	88	12	+0.00 [−0.03, +0.08]	86%	0.63	1.09	56%
	2	92	8	+0.05 [−0.00, +0.08]	89%	0.80	1.06	71%
	5	94	6	+0.05 [−0.04, +0.12]	87%	0.84	0.97	80%
	10	90	10	+0.06 [−0.11, +0.14]	87%	0.75	0.93	75%
	21	82	18	+0.15 [−0.06, +0.23]	71%	0.68	0.84	76%
14 days ($k = 0.05$)	1	98	2	+0.00 [−0.01, +0.06]	94%	0.69	0.98	68%
	2	98	2	+0.05 [−0.01, +0.15]	88%	0.76	0.90	81%
	5	94	6	+0.18 [−0.02, +0.25]	74%	0.65	0.75	78%
	10	86	14	+0.21 [−0.05, +0.43]	44%	0.48	0.64	67%
	21	80	20	+0.12 [−0.16, +0.34]	40%	0.30	0.49	52%
7 days ($k = 0.10$)	1	96	4	+0.02 [−0.02, +0.07]	92%	0.77	0.96	78%
	2	94	6	+0.03 [−0.02, +0.16]	74%	0.64	0.84	69%
	5	96	4	+0.17 [+0.04, +0.45]	52%	0.45	0.60	69%
	10	80	20	+0.20 [−0.14, +0.54]	38%	0.38	0.48	67%
	21	72	28	+0.10 [−0.03, +0.58]	47%	0.18	0.34	44%

Table 3: Trading rate against the speed of x , at edge 0.10, 50 datasets per row (the 35-day row is the edge-0.10 block of Table 2).

Reading the figures and tables.

- At edge 0.10 the tree splits on x in 81 to 99% of the datasets and puts its threshold at the mean: median 0.00 to 0.07, quartiles within ± 0.05 daily and $[-0.07, +0.25]$ monthly. Out of sample it is 0.11 to 0.23 of annual Sharpe below the ideal rule (standard error 0.03–0.04) and captures 75 to 85% of the gain. The rule at d equals the ideal rule: with these sizes the ideal position is all or nothing.
- At edge 0.05 the threshold is found in 62 to 82% of the datasets but with quartiles of about ± 0.2 , and 25 to 51% of the gain is captured. At edge 0.02 the thresholds spread over the whole range and no gain is captured; the ideal rule itself reaches a Sharpe of 0.15 at most. Fifteen years of data do not contain that signal.
- The trading rate matters little from daily to weekly. Every two weeks and monthly lose ground (81% found and 60% within ± 0.25 at $h = 21$ against 97% and 95% daily), for two reasons visible in Table 3: fewer decisions to learn from (180 monthly ones) and, when x is fast, a regime that flips inside the holding period ($g_{21}(1) = 0.87\%$ at $k = 0.10$ against 1.79% at $k = 0.02$).
- The slow rates show a small upward bias of the median threshold (+0.06 to +0.25): with a holding period and fees, entering just above the mean is not worth the trade, so the tree waits for x to be clearly positive.
- The no-split share rises from 3% (daily) to 19% (monthly) at edge 0.10 and reaches 42–46% at edge 0.02: when the checking decisions do not confirm a split, the tree stays flat, which is the safe side.

5 The first experiment (18 September): a large drift with decoys

The first run used the same tree, optimizer and price model, but with a step as the time unit and sizes far from markets: $\mu = 3\%$ per step for $\sigma = 3.2\%$ per step, an edge of about 0.95 instead of 0.02–0.10. It also placed the threshold *below* the mean of x and hid x among decoys. It is kept here because it answers two questions the new run does not: are decoys ignored, and what does the tree find when $d \neq m$?

Parameter	Value	Note
μ, σ	3%, 3.2% per step	edge 0.95; volatilities 1 to 30% per step also swept
$d; m, s^2$	0; 1, 4	d half a standard deviation below the mean: $x \geq d$ 69% of the time
k	0.2 per step	a regime lasts about 5 steps; 0.01 to 0.5 also swept
h	10 steps (and 1)	a position held while the regime changes twice on average
features	x , 20-step average of x , an unrelated OU with the same law, white noise	three decoys
data	443 training decisions (332 grow, 111 prune), 300 test	100 datasets per histogram, 20 per cell of the $k \times \sigma$ map, 800 fits

Table 4: The first experiment.

What should the tree find when $d \neq m$? Not d . Held for 10 steps, a position taken just below d is soon above it, because x reverts to $m = 1 > d$. The expected 10-step return g_{10} of (5) changes sign at $x \approx -0.92$, and the ideal position rises from 0 to 1 between there and about -0.3 . The right question was “does the tree put its threshold in $[-0.9, -0.3]$?”, and d itself is the target only for $h = 1$.

Results.

- $h = 1$ (target d): first split on x in 100 datasets of 100, one split per tree, never a decoy; median threshold 0.000, 95 of 100 within 0.1 of d ($s = 2$). Test Sharpe per decision: tree 0.684, ideal rule 0.682, always the asset 0.270.
- $h = 10$ (target $[-0.9, -0.3]$): first split on x in 86 of 100, never first on a decoy, no split in 14; median -0.46 , 90% between -1.53 and $+0.01$. Test Sharpe per decision: tree 0.724, ideal 0.742, rule at d 0.749, always the asset 0.628 (tree -0.018 ± 0.004 against the ideal rule). Fifteen trees kept a second split on x , three a second-level split on the noise decoy.
- $k \times \sigma$ map ($h = 10$, 600 trees): x is found first in 95–100% of the datasets when a regime lasts at least about one holding period ($k \leq 0.1$) and $\sigma \leq 20\%$; at $k = 0.5$ (a regime of 2 steps) in 20–45%, and then the usual outcome is no split, which is right (the ideal rule gains 0.01% per decision). Over the 600 trees the noise decoy is the first split in 9 (1.5%) and the moving average in 30 (5%), almost all at $\sigma \geq 20\%$.
- The sign change of g_{10} moves with k ($-0.04, -0.05, -0.16, -0.37, -0.92, -3.6$ for $k = 0.01$ to 0.5) and the tree’s median thresholds follow it ($-0.02, +0.02, -0.04, -0.19, -0.47, -1.52$), staying between the sign change and d .

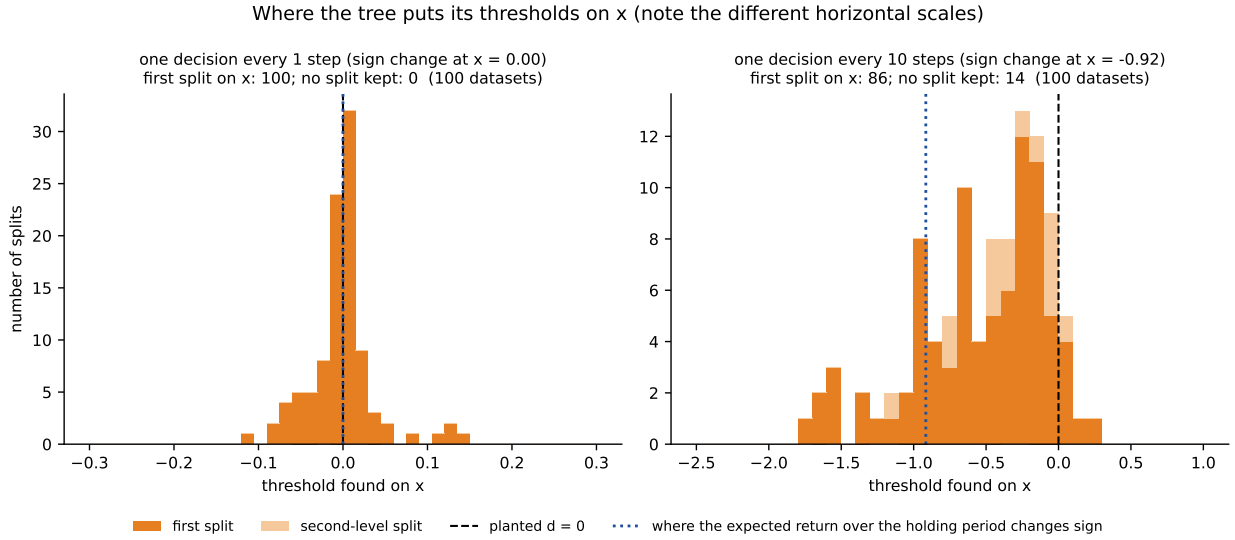


Figure 8: First experiment: thresholds kept, 100 datasets per panel. *Left:* $h = 1$, target d . *Right:* $h = 10$, target between the sign change (dotted) and about -0.3 .

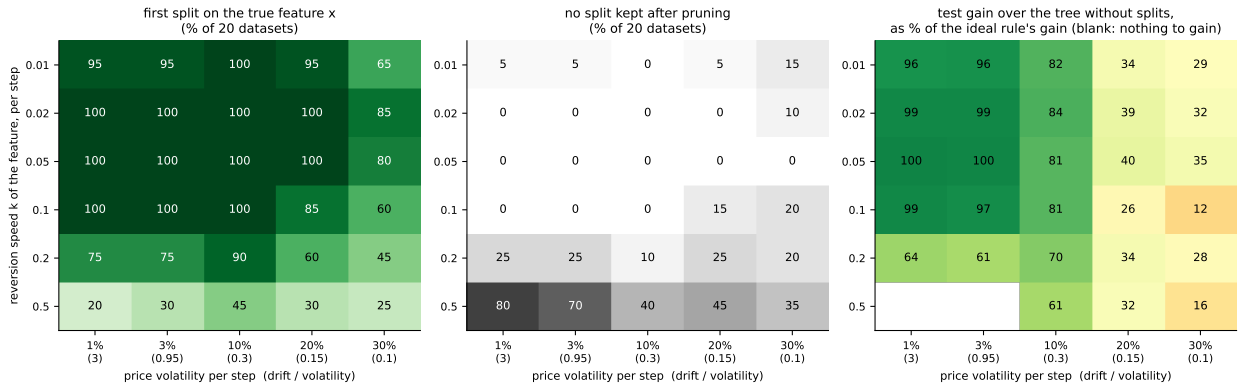


Figure 9: First experiment, $h = 10$, 20 datasets per cell: rows are the speed k of x , columns the volatility of the price (edge in brackets).

Why it was not enough. An edge of 0.95 means the regime is visible in a handful of decisions; the experiments of the main report on real data worked near 0.07. The present run repeats the test at 0.02–0.10, which is what connects it to markets.

6 What changed in the code

- **recovery_test.py:** the time unit is the trading day (252 a year); the tree sees x alone; $d = m$; the training and test spans are given in days (15 and 5 years), so that every trading rate sees the same history; the drift is given as an edge $e = \mu/\sigma$; results are annualized by $252/h$.
- **recovery_figures.py:** the sweeps are trading rate $\times$ edge and trading rate $\times$ speed; the maps carry the annualized Sharpe ratios; a summary table is printed by `plot`.
- **main.py**, one definition: the Sharpe ratio of a path with fewer than two periods stays undefined, but a path with constant zero net returns (all cash, no trade) now has a Sharpe ratio of 0 instead of NaN. Every comparison “Sharpe_{with split} > Sharpe_{without}” was false when the tree without the split sat in cash, so pruning removed every such split, however good; with a zero-mean asset this happened in most datasets. A unit test covers it.

7 What this test says, and what it does not

- **The mechanism works at market sizes.** With a regime drift of $\pm 25\%$ a year and a feature that persists for a month, the tree recovers the planted threshold at every trading rate and delivers most of the attainable Sharpe ratio out of sample. At $\pm 13\%$ it degrades gracefully; at $\pm 5\%$ it correctly finds nothing.
- **Deciding more often does not teach more about a slow signal** (daily to weekly are equivalent), but deciding too rarely loses the fast part of the signal and the number of decisions to learn from.
- **Tail leaves.** With 3,780 daily rows the minimum of 20 decisions per leaf is 0.7% of the data, and pruning lets some second-level splits in the tails through (the bars at ± 1.5 of Figure 4); they cost little here but the minimum leaf size should scale with the number of rows.
- **Not shown:** decoys at these edges, an unconditional premium of the asset, several assets, a regime depending on more than one threshold, volatility that changes with the regime.

8 How to reproduce

Code and results are in `PaperAlgo`; the results were produced with `synthetic_data.py` as committed in `eae145c` (one volatility for both regimes).

- `python recovery_test.py`: one dataset, printed in detail (about 10 s).
- `python recovery_figures.py run`: the 2,000 fits in parallel, 103 minutes on 10 cores; writes `report/figures/recovery_results.json`.
- `python recovery_figures.py plot`: the figures and the summary table of Tables 2–3.
- `python -m unittest discover -s tests`: 52 tests, including the alignment and no-look-ahead checks of the rows and the Sharpe-of-cash convention.